SIMATS ENGINEERING

SIMATS Online Programming Lab

JAVA PROGRAMMING

NITHISH RAAJU V
192411016RD

Q1
Multiple Threads Using a Single Class
10 marks

Q2
Multiple thread classes
20 marks

Q3
Thread Synchronization
10 marks

3/3 answered

opic: synchronized, shared resource, race condition

Question:
Write a Java program to demonstrate thread synchronization using a shared Counter object.

Create a class Counter containing an integer variable count, initially set to 0.

Create two threads, ThreadA and ThreadB. Both threads should increment the same count variable 1000 times.

Implement the increment() method using the synchronized keyword so that only one thread can modify the counter at a time.

After both threads complete their execution, display the final value of count.

Expected Output:

ThreadA completed
ThreadB completed
Final Count = 2000

Requirements:

Use a shared Counter object.
Both threads must access the same Counter object.
Use synchronized for the increment operation.

Use join() to ensure both threads finish before displaying the final count.

Your Code

```
public class Main {
    public static void main(String[] args)
        Counter counter = new Counter();

        ThreadA a = new ThreadA(counter);
        ThreadB b = new ThreadB(counter);

        a.start();
        b.start();

        a.join();
        b.join();

        System.out.println("Final Count =
    }
}

class Counter {
    private int count = 0;

    public synchronized void increment()
        count++;
    }

    public int getCount() {
        return count;
    }
}

class ThreadA extends Thread {
    private Counter counter;

    ThreadA(Counter counter) {
        this.counter = counter;
    }

    public void run() {
        for (int i = 0; i < 1000; i++) {
            counter.increment();
        }
        System.out.println("ThreadA comple
    }
```

```
43 }
44
45 class ThreadB extends Thread {
46     private Counter counter;
47
48     ThreadB(Counter counter) {
49         this.counter = counter;
50     }
51
52     public void run() {
53         for (int i = 0; i < 1000; i++) {
54             counter.increment();
55         }
56         System.out.println("ThreadB comple
57     }
58 }
```

Input

stdin input

Output

```
ThreadA completed
ThreadB completed
Final Count = 2000
```

Visible Test Cases

Test Case 1 ✓

Test Case 2 ✓

Test Case 3 ✓

10.00 / 10 marks

Submitted